

Robot Keyboard Control

Robot Keyboard Control

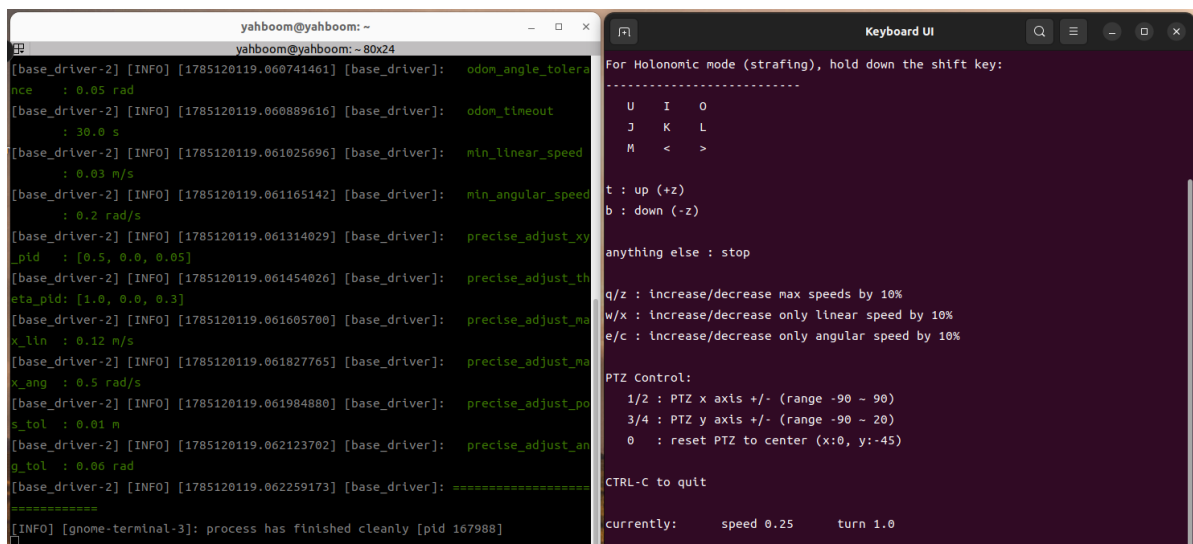
1. Course Content
2. Preparation
3. Key Operations
 - 3.1 Chassis Motion Control
 - 3.2 Speed Adjustment
 - 3.3 Gimbal (PTZ) Control
4. Source Code Analysis
 - 4.1 Key Mapping Dictionary
 - 4.2 Node Class `KeyBoardControlNode`
 - 4.3 Gimbal Control Methods
 - 4.4 Keyboard Reading Helper Functions
 - 4.5 Main Function `main` Flow

1. Course Content

- Use the keyboard to control the robot chassis and the gimbal camera
- Understand the internal implementation source code of keyboard control

2. Preparation

```
ros2 launch microros_v2_bringup controller.launch.py
```



3. Key Operations

[!IMPORTANT]

During keyboard control, keep the **Keyboard UI** terminal window focused and switch the input method to English, otherwise the keys will not take effect.

3.1 Chassis Motion Control

Key	Function
u	Move forward-left (forward + turn left)
i	Move forward
o	Move forward-right (forward + turn right)
j	Turn left in place
k	Stop
l	Turn right in place
m	Move backward-left (backward + turn left)
,	Move backward
.	Move backward-right (backward + turn right)

3.2 Speed Adjustment

Key	Function
q	Increase both the max linear velocity and max angular velocity by 10%
z	Decrease both the max linear velocity and max angular velocity by 10%
w	Increase only the linear velocity by 10%
x	Decrease only the linear velocity by 10%
e	Increase only the angular velocity by 10%
c	Decrease only the angular velocity by 10%

3.3 Gimbal (PTZ) Control

Key	Function
1	Increase the gimbal X-axis angle (range -90 ~ 90)
2	Decrease the gimbal X-axis angle (range -90 ~ 90)
3	Increase the gimbal Y-axis angle (range -90 ~ 20)
4	Decrease the gimbal Y-axis angle (range -90 ~ 20)
0	Reset the gimbal to the center position (x:0, y:-45)

[!NOTE]

To turn off keyboard control, press ctrl+c in the opened terminal to end the process and close the terminal window

4. Source Code Analysis

- Source code path

```
$HOME/microros_ws/src/common_demo/common_demo/control/keyboard_control.py
```

4.1 Key Mapping Dictionary

At the beginning of the source code, two dictionaries are defined to map keyboard keys to the robot's motion control quantities:

```
# Motion control key mapping: (x, y, z, th) correspond to the forward/backward,
left/right, up/down, and yaw direction components
moveBindings = {
    'i': (1, 0, 0, 0),    # forward
    ',': (-1, 0, 0, 0),   # backward
    'j': (0, 0, 0, 1),    # turn left in place
    'l': (0, 0, 0, -1),   # turn right in place
    'u': (1, 0, 0, 1),    # forward-left diagonal
    'o': (1, 0, 0, -1),   # forward-right diagonal
    'm': (-1, 0, 0, 1),   # backward-left diagonal
    '.'': (-1, 0, 0, -1), # backward-right diagonal
}

# Speed adjustment key mapping: (linear velocity multiplier, angular velocity
multiplier)
speedBindings = {
    'q': (1.1, 1.1),    # increase both by 10%
    'z': (0.9, 0.9),    # decrease both by 10%
    'w': (1.1, 1.0),    # increase only linear velocity by 10%
    'x': (0.9, 1.0),    # decrease only linear velocity by 10%
    'e': (1.0, 1.1),    # increase only angular velocity by 10%
    'c': (1.0, 0.9),    # decrease only angular velocity by 10%
}
```

After a key is pressed, the program multiplies the direction components in the dictionary by the current `speed` / `turn` parameters to generate the final `Twist` velocity command.

4.2 Node Class `KeyboardControlNode`

`KeyboardControlNode` inherits from ROS 2's `Node` and is responsible for parameter management, topic publishing, and gimbal control.

```

class KeyboardControlNode(Node):
    def __init__(self):
        super().__init__('keyboard_control_node')

        # Declare parameters with defaults
        self.declare_parameter('speed', 0.25)
        self.declare_parameter('turn', 1.0)
        self.declare_parameter('ptz_step', 2.0)    # degrees per key press

        # Get parameters
        self.speed = self.get_parameter('speed').value
        self.turn = self.get_parameter('turn').value
        self.ptz_step = self.get_parameter('ptz_step').value

```

- **Parameter declaration:** Three adjustable parameters are registered via `declare_parameter`, which can be overridden at startup through the launch file:
 - `speed`: Max linear velocity (m/s), default 0.25
 - `turn`: Max angular velocity (rad/s), default 1.0
 - `ptz_step`: Gimbal angle change per key press (degrees), default 2.0

```

# PTZ limits and center position (deg)
self.ptz_x_bounds = (-90.0, 90.0)
self.ptz_y_bounds = (-90.0, 20.0)
self.ptz_center = (0.0, -45.0)

# Current PTZ angles, start from center position
self.ptz_x = self.ptz_center[0]
self.ptz_y = self.ptz_center[1]

self.x = 0
self.y = 0
self.z = 0
self.th = 0

```

- **Gimbal limits and initial values:** Sets the valid angle ranges of the X/Y axes and the center position (0, -45), ensuring that the later `adjust_ptz` will not go out of bounds.
- **Motion state variables:** `x/y/z/th` store the direction components corresponding to the current key press, updated after each key press.

```

# Create qos profile for reliable communication
qos_profile = QoSProfile(
    reliability=ReliabilityPolicy.RELIABLE,
    history=HistoryPolicy.KEEP_LAST,
    depth=10
)

# Publishers
self.cmd_vel_pub = self.create_publisher(Twist, '/cmd_vel', qos_profile)
self.cmd_ptz_angle_pub = self.create_publisher(Vector3,
'/cmd_ptz_angle', 10)

```

- **Publishers:**
 - `/cmd_vel` (`geometry_msgs/Twist`): Chassis velocity commands, using a reliable QoS to ensure commands are not lost

- `/cmd_ptz_angle` (`geometry_msgs/Vector3`): Gimbal target angle commands (unit: degrees), x is the horizontal angle, y is the pitch angle

4.3 Gimbal Control Methods

```
def publish_ptz(self):
    """Publish current PTZ target angle (deg)"""
    msg = Vector3()
    msg.x = float(self.ptz_x)
    msg.y = float(self.ptz_y)
    msg.z = 0.0
    self.cmd_ptz_angle_pub.publish(msg)

def adjust_ptz(self, dx, dy):
    """Adjust PTZ angle with bounds checking"""
    self.ptz_x = max(self.ptz_x_bounds[0], min(self.ptz_x_bounds[1],
self.ptz_x + dx))
    self.ptz_y = max(self.ptz_y_bounds[0], min(self.ptz_y_bounds[1],
self.ptz_y + dy))
    self.publish_ptz()

def reset_ptz(self):
    """Reset PTZ to center position"""
    self.ptz_x, self.ptz_y = self.ptz_center
    self.publish_ptz()
```

- `publish_ptz`: Packs the current PTZ angles into a `vector3` message and publishes it.
- `adjust_ptz`: Adjusts the angle with bounds checking, using `max/min` to ensure the limits are not exceeded.
- `reset_ptz`: Resets the gimbal to the center position `(0, -45)`.

4.4 Keyboard Reading Helper Functions

```
def getKey():
    tty.setraw(sys.stdin.fileno())
    select.select([sys.stdin], [], [], 0)
    key = sys.stdin.read(1)
    termios.tcsetattr(sys.stdin, termios.TCSADRAIN, settings)
    return key

def vels(speed, turn):
    return "currently:\tspeed %s\tturn %s " % (speed, turn)
```

- `getKey`: Temporarily switches the terminal to raw mode, reads a single key non-blockingly, and then restores the terminal attributes, ensuring immediate key response without affecting the terminal display.
- `vels`: Formats and outputs the current velocity parameters for real-time viewing in the terminal.

4.5 Main Function `main` Flow

```
def main(args=None):
    global settings
    if args is None:
        args = sys.argv

    # keyboard control requires a real terminal (stdin as TTY)
    if not sys.stdin.isatty():
        print("[keyboard_control] stdin is not a TTY, keyboard control needs a
real terminal.")
        print("[keyboard_control] Run it with: ros2 run common_demo
keyboard_control")
        sys.exit(1)
    settings = termios.tcgetattr(sys.stdin)
```

- **TTY detection:** Keyboard control depends on a real terminal. If it is started in non-interactive mode via `ros2 launch`, it exits directly and prints the correct usage.
- **Save terminal attributes:** `termios.tcgetattr` saves the original terminal state for restoration in `finally`.

```
rcipy.init(args=args)
keyboard_node = KeyBoardControlNode()

try:
    # Move PTZ to center position on startup
    time.sleep(0.5) # wait for publisher connection
    keyboard_node.reset_ptz()
    print(msg)
    print(vels(keyboard_node.speed, keyboard_node.turn))
```

- Initialize ROS 2 and create the node.
- On startup, wait 0.5 seconds for the publishers to establish connections, then reset the gimbal to the center position, and print the key prompts and the current velocity parameters.

```
status = 0
while rcipy.ok():
    key = getKey()
    if key in moveBindings.keys():
        keyboard_node.x = moveBindings[key][0]
        keyboard_node.y = moveBindings[key][1]
        keyboard_node.z = moveBindings[key][2]
        keyboard_node.th = moveBindings[key][3]
    elif key in speedBindings.keys():
        keyboard_node.speed = keyboard_node.speed * speedBindings[key]
[0]
        keyboard_node.turn = keyboard_node.turn * speedBindings[key][1]

        print(vels(keyboard_node.speed, keyboard_node.turn))
        if (status == 14):
            print(msg)
            status = (status + 1) % 15
    elif key == '1':
        # PTZ x axis increase
```

```

        keyboard_node.adjust_ptz(keyboard_node.ptz_step, 0.0)
    elif key == '2':
        # PTZ x axis decrease
        keyboard_node.adjust_ptz(-keyboard_node.ptz_step, 0.0)
    elif key == '3':
        # PTZ y axis increase
        keyboard_node.adjust_ptz(0.0, keyboard_node.ptz_step)
    elif key == '4':
        # PTZ y axis decrease
        keyboard_node.adjust_ptz(0.0, -keyboard_node.ptz_step)
    elif key == '0':
        # Reset PTZ to center position
        keyboard_node.reset_ptz()
    else:
        keyboard_node.x = 0
        keyboard_node.y = 0
        keyboard_node.z = 0
        keyboard_node.th = 0
        if (key == '\x03'):
            break

```

Main loop logic:

1. Read the key → look up `moveBindings` to update the direction components.
2. If it is a speed adjustment key → modify `speed` / `turn` by the multiplier, refreshing the key prompts every 15 times.
3. If it is `1/2/3/4/0` → call the gimbal control method.
4. For any other key → clear the motion components; if it is `Ctrl+C` (`\x03`), exit the loop.

```

twist = Twist()
twist.linear.x = keyboard_node.x * keyboard_node.speed
twist.linear.y = keyboard_node.y * keyboard_node.speed
twist.linear.z = keyboard_node.z * keyboard_node.speed
twist.angular.x = 0.0
twist.angular.y = 0.0
twist.angular.z = keyboard_node.th * keyboard_node.turn
keyboard_node.cmd_vel_pub.publish(twist)

```

- Multiply the direction components by the current velocity parameters, assemble them into a `Twist` message, and publish it to `/cmd_vel`.

```

except Exception as e:
    keyboard_node.get_logger().error(f"Error: {e}")

finally:
    twist = Twist()
    twist.linear.x = 0.0
    twist.linear.y = 0.0
    twist.linear.z = 0.0
    twist.angular.x = 0.0
    twist.angular.y = 0.0
    twist.angular.z = 0.0
    keyboard_node.cmd_vel_pub.publish(twist)

    termios.tcsetattr(sys.stdin, termios.TCSADRAIN, settings)

```

```
keyboard_node.destroy_node()
rcipy.shutdown()
```

```
class KeyboardControlNode(Node):
    def __init__(self):
        super().__init__('keyboard_control_node')

        # Declare parameters with defaults
        self.declare_parameter('speed', 0.25)
        self.declare_parameter('turn', 1.0)
        self.declare_parameter('ptz_step', 2.0)    # degrees per key press

        # Get parameters
        self.speed = self.get_parameter('speed').value
        self.turn = self.get_parameter('turn').value
        self.ptz_step = self.get_parameter('ptz_step').value

        # PTZ limits and center position (deg)
        self.ptz_x_bounds = (-90.0, 90.0)
        self.ptz_y_bounds = (-90.0, 20.0)
        self.ptz_center = (0.0, -45.0)

        # Current PTZ angles, start from center position
        self.ptz_x = self.ptz_center[0]
        self.ptz_y = self.ptz_center[1]

        self.x = 0
        self.y = 0
        self.z = 0
        self.th = 0

        # Create QoS profile for reliable communication
        qos_profile = QoSProfile(
            reliability=ReliabilityPolicy.RELIABLE,
            history=HistoryPolicy.KEEP_LAST,
            depth=10
        )

        # Publishers
        self.cmd_vel_pub = self.create_publisher(Twist, '/cmd_vel', qos_profile)
        self.cmd_ptz_angle_pub = self.create_publisher(Vector3,
            '/cmd_ptz_angle', 10)

    def publish_ptz(self):
        """Publish current PTZ target angle (deg)"""
        msg = Vector3()
        msg.x = float(self.ptz_x)
        msg.y = float(self.ptz_y)
        msg.z = 0.0
        self.cmd_ptz_angle_pub.publish(msg)

    def adjust_ptz(self, dx, dy):
        """Adjust PTZ angle with bounds checking"""
        self.ptz_x = max(self.ptz_x_bounds[0], min(self.ptz_x_bounds[1],
            self.ptz_x + dx))
```



```

        self.ptz_y = max(self.ptz_y_bounds[0], min(self.ptz_y_bounds[1],
self.ptz_y + dy))
        self.publish_ptz()

    def reset_ptz(self):
        """Reset PTZ to center position"""
        self.ptz_x, self.ptz_y = self.ptz_center
        self.publish_ptz()

def getKey():
    tty.setraw(sys.stdin.fileno())
    select.select([sys.stdin], [], [], 0)
    key = sys.stdin.read(1)
    termios.tcsetattr(sys.stdin, termios.TCSADRAIN, settings)
    return key

def vels(speed, turn):
    return "currently:\tspeed %s\tturn %s " % (speed, turn)

def main(args=None):
    global settings
    if args is None:
        args = sys.argv

    # keyboard control requires a real terminal (stdin as TTY)
    if not sys.stdin.isatty():
        print("[keyboard_control] stdin is not a TTY, keyboard control needs a
real terminal.")
        print("[keyboard_control] Run it with: ros2 run common_demo
keyboard_control")
        sys.exit(1)
    settings = termios.tcgetattr(sys.stdin)

    rclpy.init(args=args)
    keyboard_node = KeyBoardControlNode()

    try:
        # Move PTZ to center position on startup
        time.sleep(0.5) # wait for publisher connection
        keyboard_node.reset_ptz()
        print(msg)
        print(vels(keyboard_node.speed, keyboard_node.turn))

    status = 0
    while rclpy.ok():
        key = getKey()
        if key in moveBindings.keys():
            keyboard_node.x = moveBindings[key][0]
            keyboard_node.y = moveBindings[key][1]
            keyboard_node.z = moveBindings[key][2]
            keyboard_node.th = moveBindings[key][3]
        elif key in speedBindings.keys():
            keyboard_node.speed = keyboard_node.speed * speedBindings[key]

[0]
            keyboard_node.turn = keyboard_node.turn * speedBindings[key][1]

            print(vels(keyboard_node.speed, keyboard_node.turn))
            if (status == 14):

```

```

        print(msg)
        status = (status + 1) % 15
    elif key == '1':
        # PTZ x axis increase
        keyboard_node.adjust_ptz(keyboard_node.ptz_step, 0.0)
    elif key == '2':
        # PTZ x axis decrease
        keyboard_node.adjust_ptz(-keyboard_node.ptz_step, 0.0)
    elif key == '3':
        # PTZ y axis increase
        keyboard_node.adjust_ptz(0.0, keyboard_node.ptz_step)
    elif key == '4':
        # PTZ y axis decrease
        keyboard_node.adjust_ptz(0.0, -keyboard_node.ptz_step)
    elif key == '0':
        # Reset PTZ to center position
        keyboard_node.reset_ptz()
    else:
        keyboard_node.x = 0
        keyboard_node.y = 0
        keyboard_node.z = 0
        keyboard_node.th = 0
        if (key == '\x03'):
            break

    twist = Twist()
    twist.linear.x = keyboard_node.x * keyboard_node.speed
    twist.linear.y = keyboard_node.y * keyboard_node.speed
    twist.linear.z = keyboard_node.z * keyboard_node.speed
    twist.angular.x = 0.0
    twist.angular.y = 0.0
    twist.angular.z = keyboard_node.th * keyboard_node.turn
    keyboard_node.cmd_vel_pub.publish(twist)

except Exception as e:
    keyboard_node.get_logger().error(f"Error: {e}")

finally:
    twist = Twist()
    twist.linear.x = 0.0
    twist.linear.y = 0.0
    twist.linear.z = 0.0
    twist.angular.x = 0.0
    twist.angular.y = 0.0
    twist.angular.z = 0.0
    keyboard_node.cmd_vel_pub.publish(twist)

    termios.tcsetattr(sys.stdin, termios.TCSADRAIN, settings)

    keyboard_node.destroy_node()
    rclpy.shutdown()

```